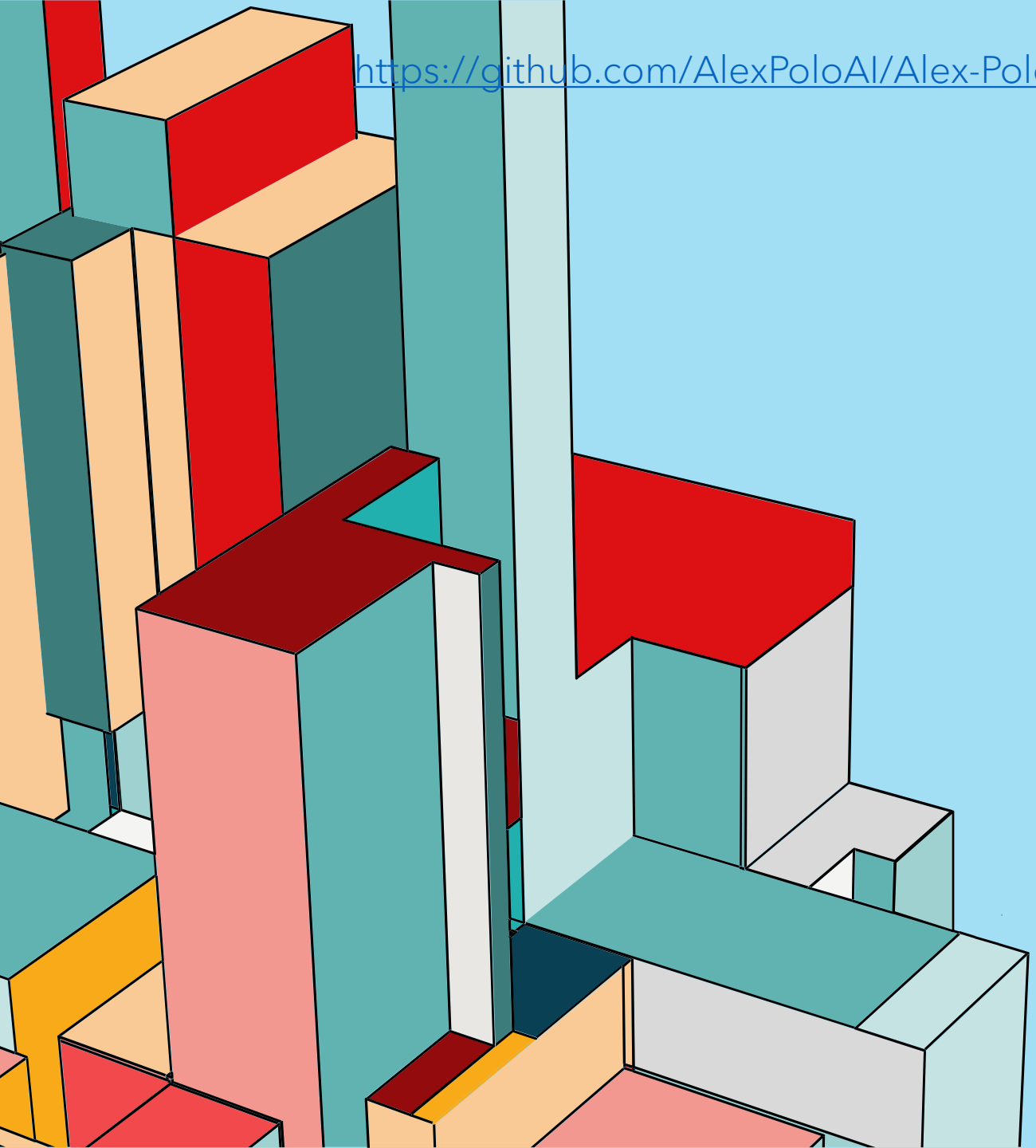
https://github.com/AlexPoloAI/Alex-Polo-AI-Portfolio/tree/main/ITAI_1378_Labs/Attractor_Project



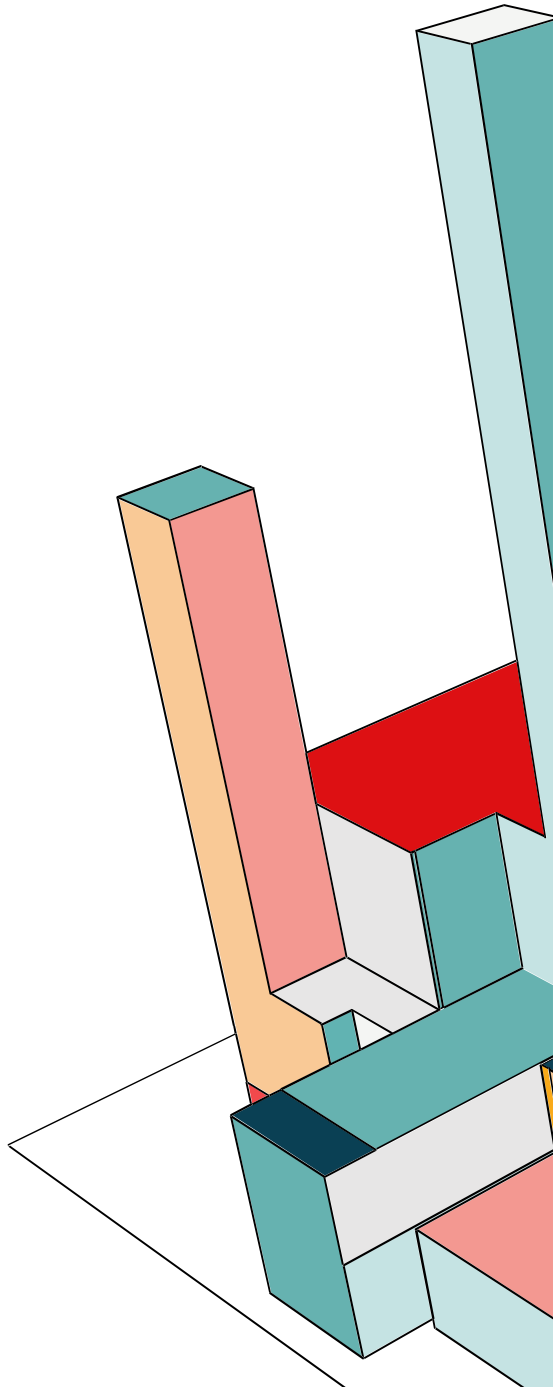
PROJECT NAME: ATTRACTOR PROJECT

DEVELOPER: ALEX POLOZOV

TIER: 1

THE PROBLEM

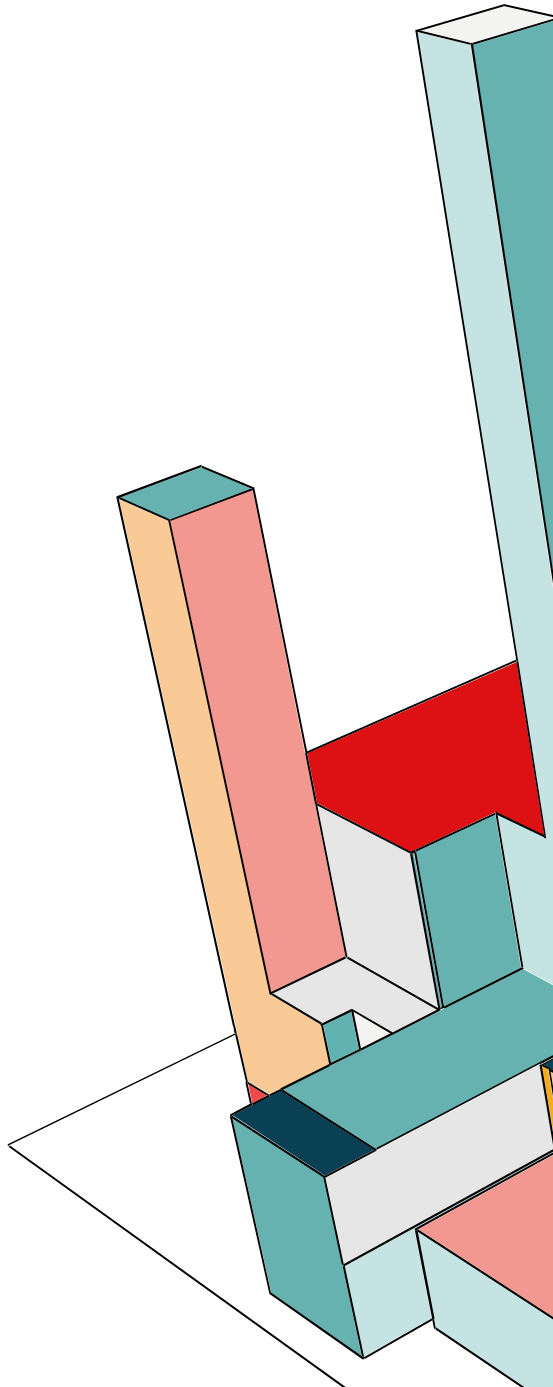
- Generic algorithms cannot recognize subconscious human taste preferences. Standard biometric sorting fails when evaluating subjective aesthetic parameters in complex visual environments.



WHAT I BUILT

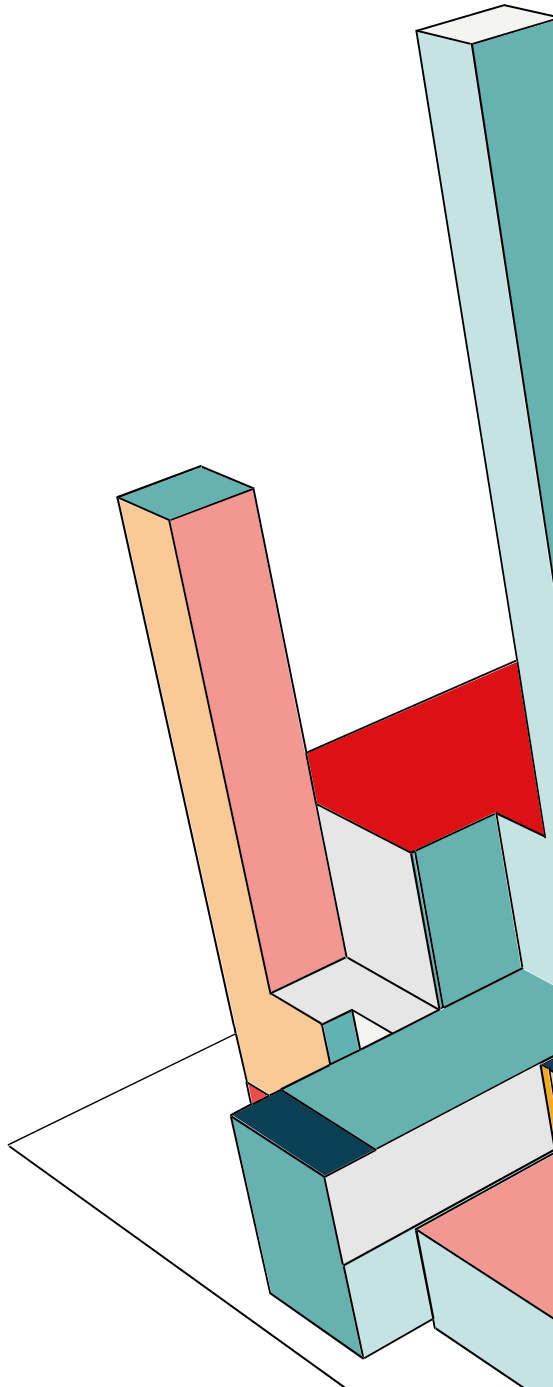
- I built a dual-pipeline computer vision system that filters images based on my personal visual taste matrix using both semantic and geometric approaches.
- System Flow:

1. YOLO11x extracts humans from raw images to create local crops.
2. JSON Pipeline: Qwen2-VL generates a semantic JSON matrix of physical and contextual attributes, scored by probability matching.
3. SVM Pipeline: ResNet50 extracts feature vectors, and One-Class SVM measures geometric distances to rank images.



DEMO

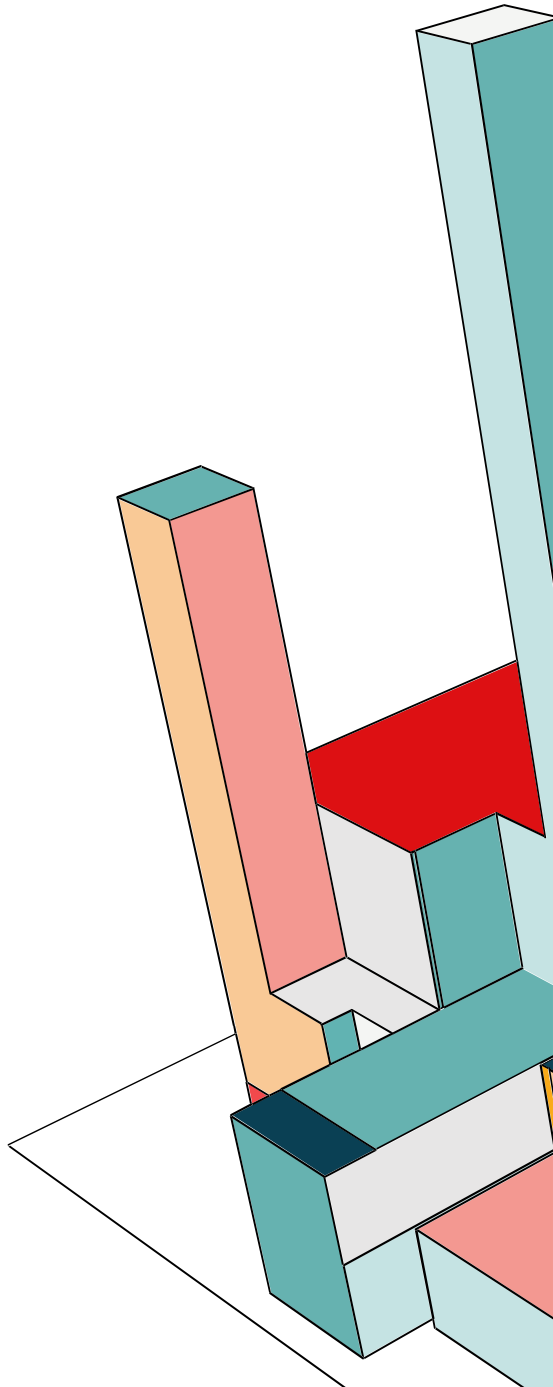
- You can watch my system work here. I recorded this video to explain my project.
- https://drive.google.com/file/d/1-eEZxfBNKs6GbjEF30dIdnxESrmOS5fd/view
- I did not run one live test in this video. How you can see: JSON pipeline needs much time to process photos, but it gives better results.
- This video is longer than 5 minutes because I want to explain all project details. I speak slowly, so you can watch this video at double speed.



HOW IT WORKS

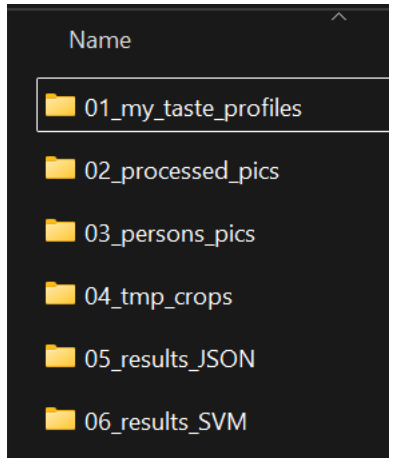
- **Detector:** YOLO11x (CUDA optimized).
- **Semantic Engine (JSON Pipeline):** Qwen2-VL-7B-Instruct (Multimodal LLM) for descriptive taxonomy mapping.
- **Geometric Engine (SVM Pipeline):** ResNet50 for feature extraction + One-Class SVM (scikit-learn) for boundary classification.
- **Logic:** The JSON pipeline converts pixels into a structured semantic taxonomy (Tier 1 to Tier 4) and scores against a normalized probability matrix, while the SVM pipeline maps image tensors into a multi-dimensional feature space to calculate hyperplane distances.

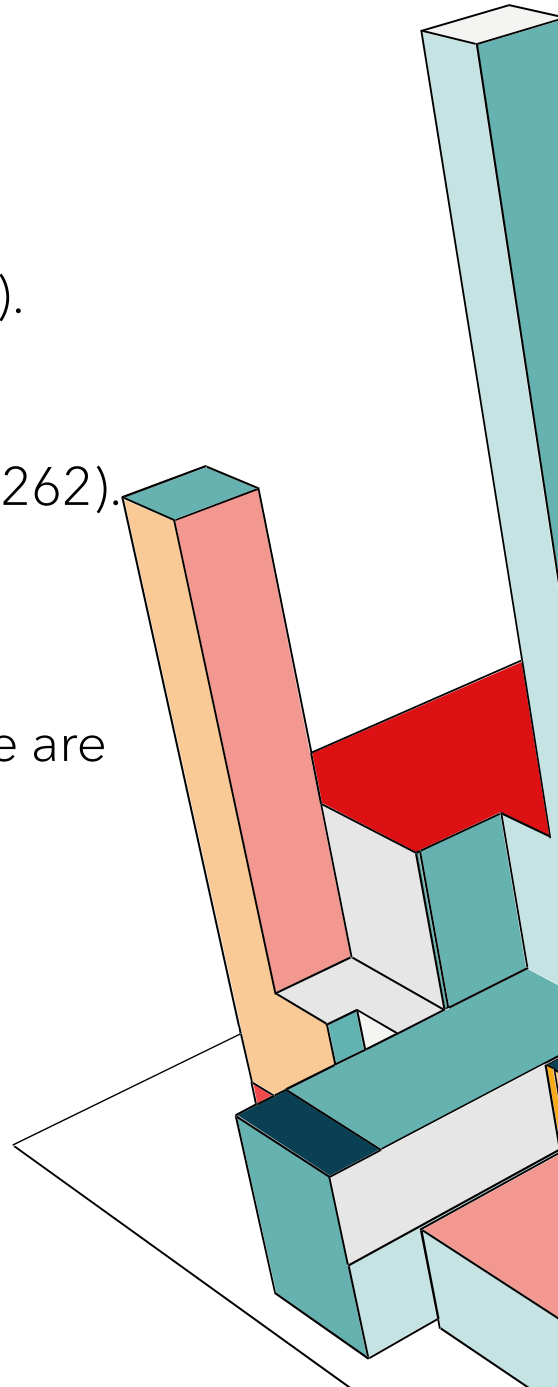
```
RUN_JSON_PIPELINE = True
RUN_SVM_PIPELINE = True
✓ 0.0s
```



THE DATA I USED

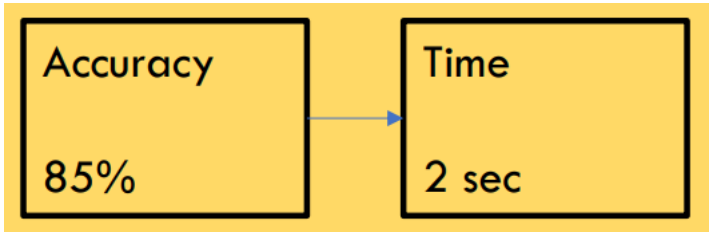
- **Source:** Private photo collection + Synthetic AI generations (Nano Banana).
- Train (01_my_taste_profiles): 50 private photos
- Test (04_tmp_crops):10 Target Images, 252 Synthetic Noise Images (Total 262).
- **Labels:** Unsupervised taste clustering (no manual pixel labeling).
- **Cleaning:** Automated filtering using YOLO11x to remove frames without humans and splitting a single image into several images if multiple people are detected



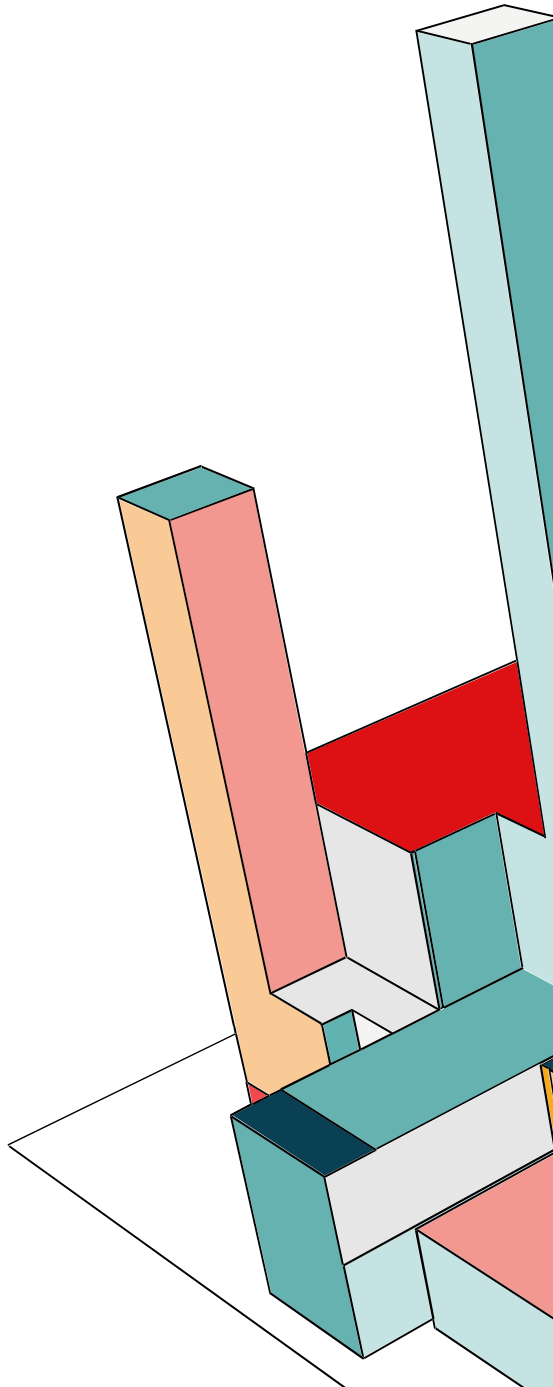


RESULTS VS MY TARGETS

Metric	Blueprint Target	Achieved (JSON Pipeline)	Achieved (SVM Pipeline)
Accuracy	>85.0%	93.89%	92.36%
Recall		70.00%	50.00%
Precision (Optimized)	100.00%	100.00% (Threshold >= 95%)	100.00% (Threshold > 0.05)

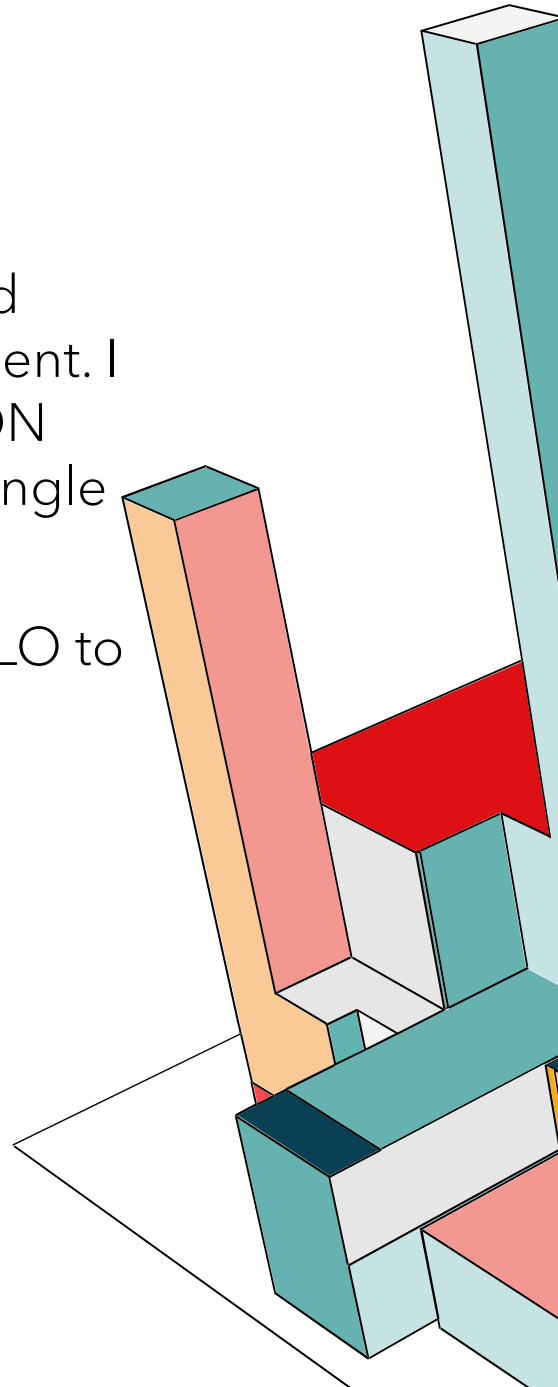


Pipeline	Metric	Value	Calculation	Description
System_Performance	Target_Time	< 2.0 sec	Goal for 1 crowd photo	Blueprint requirement for commercial viability
System_Performance	Batch_Scoring_Time	1.7 sec	1.7s (SVM only)	Execution time to score and rank 262 pre-processed crops



WHAT CHANGED FROM BLUEPRINT

- In that blueprint, I planned to use four different levels for analysis: face and small accessories, body shape, clothes and big accessories, and environment. I changed this plan. I decided to replace those four levels with one big JSON array of features. For that SVM pipeline, I combined all features into one single cloud.
- I planned to test YOLO and maybe SAM 2. I did not use SAM 2. I used YOLO to crop images. I used Qwen2-VL to generate JSON data. I used two other technologies to build that SVM pipeline: ResNet50 and scikit-learn.

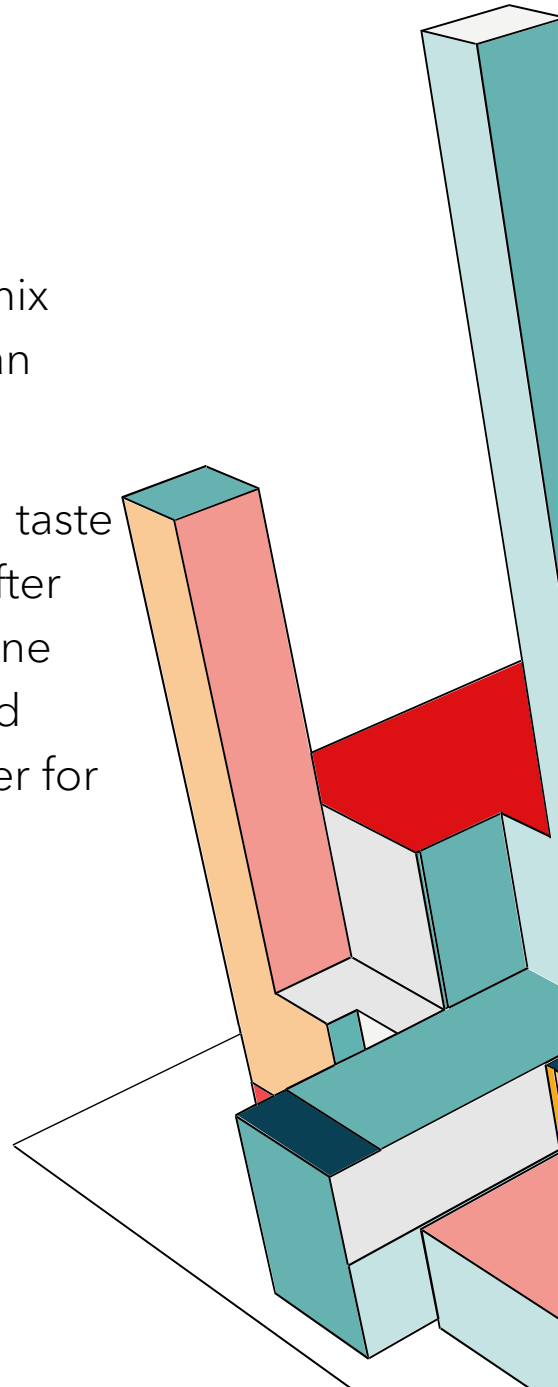


CHALLENGES AND FIXES

- **Problem:** False Positives from AI-generated images breaching the SVM hyperplane.
    - **Fix:** Shifted from pixel-distance calculation to LLM-driven semantic extraction.
- **Problem:** Hardware constraints when processing hundreds of high-res images.
    - **Fix:** Implemented GPU VRAM offloading (torch.cuda.empty_cache()) and forced matplotlib.use('Agg') backend.

LESSONS AND NEXT STEPS

- Lessons Learned: It is very good to combine computer vision with JSON parsing. This mix with SVM pipeline can make accuracy higher. Building this system was much harder than simple lab work. I had many hard bugs. I spent much time to fix them.
- Next Steps: I want to make both pipelines more accurate. I will train models not only on taste photos, but also on negative examples. I will add one new folder for wrong matches. After that step, I will build one combined JSON+SVM pipeline. I will connect this system to one web API for real-time scraping and photo sorting. I will also replace heavy YOLO11x and Qwen2-VL-7B-Instruct models with light models. This change will make processing faster for mobile devices. Finally, I want to release one commercial product.



THANK YOU FOR THIS CLASS